



Día 7

Base de Datos Segura, Docker Hardening y Reverse Proxy

Taller de Seguridad Web – OWASP Top 10

Carlos Rodríguez Contreras · GitHub: [karrocon](#)

Agenda del día

🕒 Sesión 1 (2h)

Bloque	Tema
Repaso	Warm-up día 6
T16	Base de datos segura
Lab	Mínimo privilegio SQL
T17	Docker Security
Lab	Escribir Dockerfile seguro

🕒 Sesión 2 (2h)

Bloque	Tema
T17+	Docker Compose hardening
Lab	Escanear imagen con Trivy
T18	Reverse Proxy con Nginx
Lab	Nginx + TLS + Rate Limiting

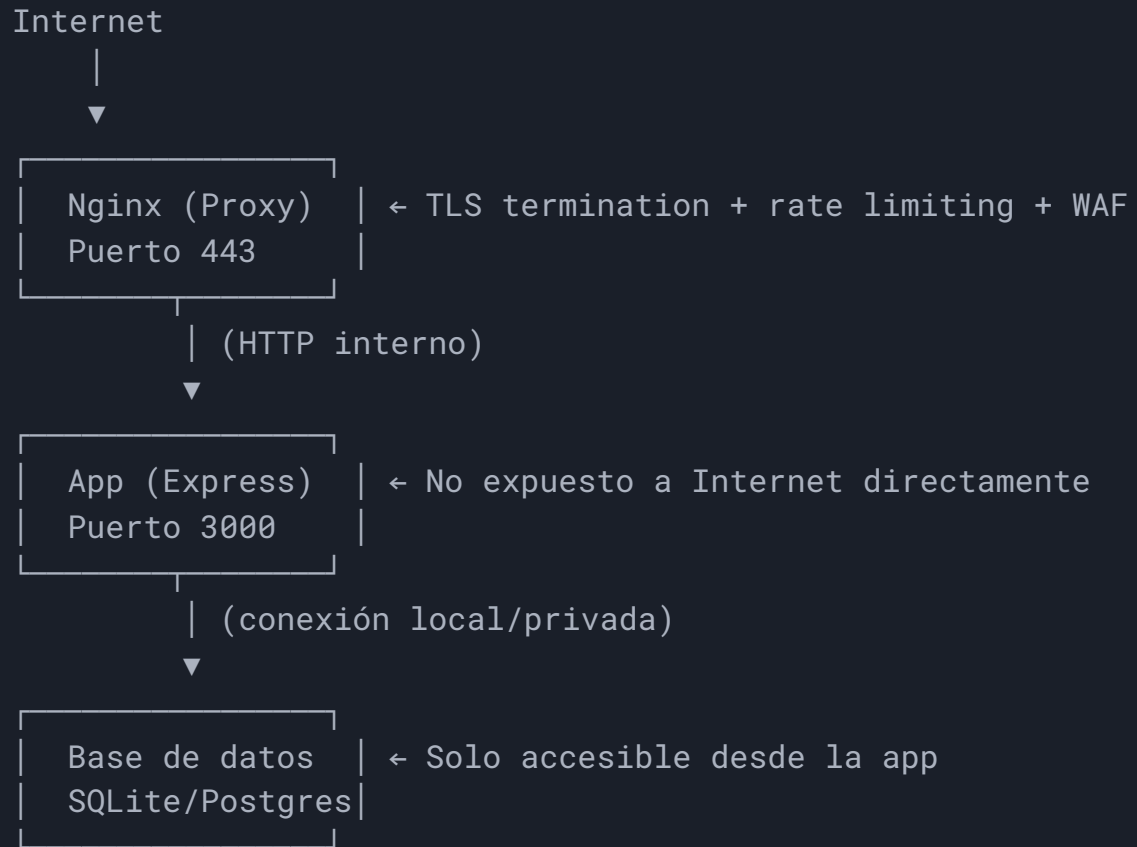
Objetivos del día

Al terminar hoy serás capaz de:

1. 🗄️ Crear usuarios SQL con **mínimo privilegio** (solo SELECT/INSERT)
2. 🐳 Escribir Dockerfiles seguros con **multi-stage** y usuario no-root
3. 🔍 Escanear imágenes Docker en busca de CVEs con **Trivy**
4. 🌐 Configurar Nginx como reverse proxy con TLS y rate limiting

💡 **Hoy es día de INFRAESTRUCTURA.** Protegemos las capas que rodean la aplicación.

Arquitectura de producción segura



T16: Base de Datos Segura

Mínimo privilegio, red y backups cifrados

Principio de mínimo privilegio SQL

-- ✗ MAL: La app usa el usuario root/admin

```
GRANT ALL PRIVILEGES ON *.* TO 'app_user'@'%';
```

-- ✓ BIEN: Solo los permisos que necesita

```
CREATE USER 'vulnapp'@'localhost' IDENTIFIED BY '...';
```

```
GRANT SELECT, INSERT ON vulnapp.users TO 'vulnapp'@'localhost';
```

```
GRANT SELECT, INSERT, UPDATE ON vulnapp.products TO 'vulnapp'@'localhost';
```

-- Sin DELETE, sin DROP, sin ALTER, sin FILE

¿Por qué? Si un atacante logra SQLi, sus opciones se limitan:

- ✗ No puede `DROP TABLE users`
- ✗ No puede `INTO OUTFILE` (leer/escribir archivos)
- ✗ No puede acceder a otras bases de datos

Aislamiento de red — expose vs ports

```
# docker-compose.yml
services:
  app:
    ports:
      - "3000:3000" # ✅ Accesible desde fuera (para el proxy)
    networks:
      - frontend
      - backend

  database:
    # ❌ NO usar ports (no exponer al host):
    # ports:
    #   - "5432:5432"
    expose:
      - "5432" # ✅ Solo accesible dentro de la red Docker
    networks:
      - backend # Solo la red backend

networks:
  frontend:
  backend:
    internal: true # Sin acceso a Internet
```

Backups cifrados con GPG

```
# Crear backup cifrado:
sqlite3 vuln.db ".backup /tmp/backup.db"
gpg --symmetric --cipher-algo AES256 /tmp/backup.db
# → genera backup.db.gpg (cifrado con passphrase)
rm /tmp/backup.db # Eliminar el no cifrado

# Para PostgreSQL:
pg_dump vulnapp | gpg --symmetric --cipher-algo AES256 > backup.sql.gpg

# Restaurar:
gpg --decrypt backup.db.gpg > /tmp/restore.db
sqlite3 vuln.db ".restore /tmp/restore.db"
```

✅ **Regla:** Todo backup debe estar cifrado. Si lo robaran, los datos serían inútiles sin la clave.

Conexión TLS a la base de datos

```
// PostgreSQL con TLS obligatorio:
const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  ssl: {
    rejectUnauthorized: true,           // Verificar certificado
    ca: fs.readFileSync('/certs/ca.pem'), // CA del servidor
  }
});

// MySQL con TLS:
const connection = mysql.createConnection({
  host: 'db-server',
  ssl: { ca: fs.readFileSync('/certs/ca-cert.pem') }
});
```

Incluso dentro de Docker: tráfico entre contenedores puede ser interceptado si la red Docker se compromete.

Lab T16 — Mínimo privilegio

Ejercicio conceptual para SQLite:

1. Abrir `src/db/database.ts` de VulnApp
2. Identificar: ¿qué operaciones hace la app sobre cada tabla?
 - `users` : SELECT (login), INSERT (registro)
 - `products` : SELECT (listado), INSERT/UPDATE/DELETE (admin)
3. ¿Qué permisos daría a un usuario `app_readonly` ?
4. ¿Y a un usuario `app_admin` ?

SQL final:

```
GRANT SELECT ON users TO app_readonly; GRANT SELECT, INSERT, UPDATE, DELETE ON products TO app_admin;
```

T17: Docker Security

Imágenes seguras, contenedores hardened

El problema – Dockerfile inseguro

```
# ❌ INSEGURO: imagen base enorme + ejecuta como root
FROM node:20
WORKDIR /app
COPY . .
RUN npm install
EXPOSE 3000
CMD ["node", "dist/index.js"]
```

Problemas:

Problema	Riesgo
<code>node:20</code> (Debian full)	~950 MB, cientos de CVEs
Sin <code>.dockerignore</code>	<code>node_modules</code> , <code>.env</code> copiados
Ejecuta como <code>root</code>	Si comprometen la app → root en contenedor
<code>npm install</code> (no <code>ci</code>)	Lockfile ignorado → versiones inesperadas

Dockerfile seguro — Multi-stage + Alpine

```
# Stage 1: Build
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
RUN npm run build

# Stage 2: Runtime (solo lo necesario)
FROM node:20-alpine
WORKDIR /app
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
USER appuser
EXPOSE 3000
CMD ["node", "dist/index.js"]
```

Comparación de tamaños de imagen

Imagen base	Tamaño	CVEs típicos
<code>node:20</code> (Debian)	~950 MB	200+
<code>node:20-slim</code>	~200 MB	50+
<code>node:20-alpine</code>	~130 MB	~5
Multi-stage (solo dist)	~80 MB	~2

✓ **Regla:** Menos software instalado = menos superficie de ataque.

Docker Compose – Hardening completo

```
services:
  app:
    image: vulnapp:latest
    read_only: true           # Filesystem solo lectura
    tmpfs:
      - /tmp                 # Solo /tmp es escribible
    cap_drop:
      - ALL                  # Eliminar TODAS las capabilities
    cap_add:
      - NET_BIND_SERVICE     # Solo lo estrictamente necesario
    security_opt:
      - no-new-privileges:true # No escalar privilegios
    mem_limit: 512m          # Limitar memoria
    cpus: 0.5                 # Limitar CPU
    environment:
      - NODE_ENV=production
    env_file:
      - .env                  # Secretos fuera del compose
```

.dockerignore — Lo que NUNCA copiar

```
# .dockerignore
node_modules
.env
.env.local
.git
.gitignore
*.md
docker-compose*.yaml
.vscode
coverage/
__tests__/
*.test.ts
```

💀 **Sin .dockerignore:** el `COPY . .` copia `.env` (con secretos) y `.git` (con historial completo) a la imagen.

Escanear imágenes con Trivy

```
# Escanear imagen local:  
trivy image vulnapp:latest
```

```
# Resultado:
```

```
# Total: 3 (HIGH: 2, CRITICAL: 1)
```

```
#  
# | Library | CVE | Severity | Fixed Version |  
# |-----|-----|-----|-----|  
# | openssl | CVE-2023-xxx | CRITICAL | 3.1.4 |  
# | libcurl | CVE-2023-yyy | HIGH | 8.4.0 |  
#
```

```
# Fallar CI si hay vulnerabilidades críticas:
```

```
trivy image --exit-code 1 --severity CRITICAL vulnapp:latest
```

Lab T17 — Dockerfile seguro

1. Crear un `Dockerfile` multi-stage para VulnApp:

- Stage 1: `node:20-alpine` + `npm ci` + `npm run build`
- Stage 2: `node:20-alpine` + `adduser` + `COPY --from=builder` + `USER appuser`

2. Crear `.dockerignore` (excluir `.env`, `node_modules`, `.git`)

3. Build y verificar:

```
docker build -t vulnapp:secure .  
docker run vulnapp:secure whoami # → appuser (no root)  
docker images vulnapp:secure    # → ~80 MB (no 950 MB)
```

 **Resultado:** imagen de ~80 MB, sin CVEs críticos, ejecutando como usuario no-root.

T18: Reverse Proxy con Nginx

TLS, Rate Limiting y WAF básico

¿Por qué un reverse proxy?

Sin proxy:

```
Internet → App (3000)
- App gestiona TLS
- App gestiona rate limiting
- App expuesta directamente
- Cada app repite config
```

Con Nginx:

```
Internet → Nginx (443) → App (3000)
- Nginx gestiona TLS
- Nginx hace rate limiting
- App no accesible directamente
- Config centralizada
```

Configuración base de Nginx

```
server {  
    listen 443 ssl http2;  
    server_name vulnapp.example.com;  
  
    # TLS moderno  
    ssl_certificate      /etc/letsencrypt/live/vulnapp/fullchain.pem;  
    ssl_certificate_key  /etc/letsencrypt/live/vulnapp/privkey.pem;  
    ssl_protocols       TLSv1.2 TLSv1.3;  
    ssl_ciphers          HIGH:!aNULL:!MD5;  
  
    # Proxy a la app  
    location / {  
        proxy_pass http://app:3000;  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
        proxy_set_header X-Forwarded-Proto $scheme;  
    }  
}
```

Rate Limiting — Frenar ataques de fuerza bruta

```
# Definir zona de rate limiting (10 req/seg por IP):
limit_req_zone $binary_remote_addr zone=login:10m rate=10r/s;
limit_req_zone $binary_remote_addr zone=api:10m rate=30r/s;

server {
    # Endpoint de login: máximo 5 req/seg con burst de 10
    location /api/login {
        limit_req zone=login burst=10 nodelay;
        limit_req_status 429; # Too Many Requests
        proxy_pass http://app:3000;
    }

    # API general: 30 req/seg
    location /api/ {
        limit_req zone=api burst=50 nodelay;
        proxy_pass http://app:3000;
    }
}
```

WAF básico con Nginx

```
# Bloquear user-agents de scanners:
if ($http_user_agent ~* (sqlmap|nikto|nmap|dirbuster|masscan)) {
    return 403;
}

# Bloquear intentos de path traversal:
location ~* \.\.(\|/|\|) {
    return 403;
}

# Bloquear extensiones peligrosas:
location ~* \.(php|asp|aspx|jsp|cgi)$ {
    return 403;
}

# Limitar tamaño de body (previene DoS):
client_max_body_size 10m;

# Ocultar versión de Nginx:
server_tokens off;
```

Let's Encrypt — TLS gratis y automático

```
# Instalar certbot:  
apt install certbot python3-certbot-nginx  
  
# Obtener certificado (automático):  
certbot --nginx -d vulnapp.example.com  
  
# Renovación automática (cron):  
certbot renew --quiet  
# certbot añade un cron automáticamente
```

Resultado: Certificado TLS válido, renovación automática cada 60 días.

Arquitectura final – docker-compose completo

```
services:
  nginx:
    image: nginx:alpine
    ports:
      - "443:443"
      - "80:80"      # Redirige a 443
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
      - ./certs:/etc/letsencrypt:ro
    depends_on:
      - app
    networks:
      - frontend

  app:
    build: .
    read_only: true
    user: "appuser"
    expose:
      - "3000"      # Solo visible dentro de Docker
    networks:
      - frontend
      - backend

  db:
    image: postgres:16-alpine
    expose:
      - "5432"      # Solo visible para app
    networks:
      - backend
```

Lab T18 — Nginx reverse proxy

1. Crear `nginx.conf` con:

- `proxy_pass http://app:3000`
- `ssl_protocols TLSv1.2 TLSv1.3`
- Rate limiting en `/api/login` (5 req/seg)
- `server_tokens off`

2. Docker compose up y verificar:

```
docker compose up -d
# Verificar rate limiting:
for i in $(seq 1 20); do
  curl -s -o /dev/null -w "%{http_code}\n" http://localhost/api/login
done
# Los últimos deben devolver 429
```

 **Resultado:** Nginx termina TLS, limita rate, oculta la app del acceso directo.

Resumen del día

Tema	Problema	Solución
BD Segura	App con permisos de admin	GRANT mínimo + red aislada + backup cifrado
Docker	Root + imagen gigante + CVEs	Multi-stage + alpine + USER + Trivy
Proxy	App expuesta directamente	Nginx + TLS + rate limit + WAF básico

🏆 **Logro del día:** Sabéis asegurar las 3 capas de infraestructura: base de datos, contenedor y red.